

SAE 2.02

Rapport final

Groupe TP 11A

DASTAN Berat et GLORO Célian

Avril 2026

1. Vue d'ensemble des fonctionnalités demandées

Fonctionnalité	Pas fait	Commencé mais pas fini	Réalisé mais non fonctionnel	Fonctionnel
Code de transformation du donjon en graphe				X
Code de résolution du problème sur le graphe avec DFS				X
Code de résolution du problème sur le graphe avec BFS				X
Code de résolution du problème sur le graphe avec A*				X
Code de transformation de la solution du graphe (liste de noeuds) en solution pour le donjon (liste de directions)				X
Ajout de 2 donjons (via le format fichier texte)				X
Analyse des résultats obtenus pour les 6 (donnés) + 2 (ajoutés) donjons.				X

Autres informations ? =>

2. Organisation du binôme

2.1. Répartition du travail

Fonctionnalité	Berat	Célian
Code de transformation du donjon en graphe	Programmation de Graph, Node et Dungeon2Graph	-
Code de résolution du problème sur le graphe avec DFS	Programmation de l'Interface Solver et de SolverGeneric	Programmation SolverWithDFS
Code de résolution du problème sur le graphe avec BFS	Programmation de l'Interface Solver et de SolverGeneric	Programmation SolverWithBFS
Code de résolution du problème sur le graphe avec A*	Programmation de l'Interface Solver, de SolverGeneric et de SolverWithA*	Programmation SolverWithA*
Code de transformation de la solution du graphe (liste de noeuds) en solution pour le donjon (liste de directions)	Programmation Dungeon2Graph et GraphSolve	-
Ajout de 2 donjons (via le format fichier texte)	Ajout du Donjon7	Ajout du 1 Donjon8
Analyse des résultats obtenus pour les 6 (donnés) + 2 (ajoutés) donjons.	Analyse Ensemble	Analyse Ensemble
Autres activités de gestion de projet	-	-

Part individuelle de travail sur l'ensemble de la SAE	50%	50%
---	------------	------------

Autres informations ? =>

3. Résultats d'analyse

3.1. Les 2 donjons supplémentaires

Nouveau donjon 1 (7ième donjon)



On a choisi d'intégrer le Donjon 7 parce que c'est une carte complètement vide et très grande. Il n'y a aucun mur à l'intérieur, c'est juste un immense espace ouvert entre le point de départ A et l'arrivée B. Le but de ce test est d'observer comment les algorithmes réagissent quand ils n'ont aucun couloir ou obstacle pour les guider. C'est le meilleur moyen de voir si un algorithme sait diriger sa recherche vers la sortie ou s'il fouille totalement au hasard.

Voici ce que ce donjon nous a permis de prouver pour les trois algorithmes :

- + A^* : C'est le seul qui est efficace ici. Comme il calcule la distance qui lui reste à parcourir, il trace une ligne droite en diagonale vers la sortie. Il va directement au but sans s'occuper de tout l'espace vide autour de lui, ce qui économise énormément de temps et de mémoire.
- + BFS : Il perd beaucoup trop de temps. Puisqu'il cherche en regardant toutes les cases autour de lui, étape par étape, il va vérifier presque toute la pièce vide dans toutes les directions avant de trouver la sortie. Ce donjon montre que sans murs pour l'arrêter, BFS fait beaucoup trop de vérifications inutiles et sature la mémoire de l'ordinateur.

+ DFS : Comme BFS, il n'est pas optimal pour ce donjon. Au lieu de traverser la pièce pour aller au plus court, son fonctionnement l'oblige à aller tout droit jusqu'à ce qu'il tape un bord de la carte, puis à longer les murs extérieurs. Ce test prouve qu'il est incapable de trouver une route intelligente dans un espace assez vaste.

Nouveau donjon 2 (8ième donjon)

```

A-o-o-o-o-o   o-o-o-o-o
|               |   |   |   |
o-o-o-o-o o-o-o-o o-o-o-o
| | | |   |
o-o o o-o-o-o-o-o-o-o-o
| | | | | | | | | |
o-o-o-o-o-o-o-o-o-o-o-o
| | | | |   | |
o-o o-o-o-o-o o-o-o-o-o
| |   |   |   |
o-o-o-o o-o-o-o-o-o-o-o-o
|   |   |   |   | |
o-o-o-o-o-o-o o-o-o-o-o
| | | | | | | | | |
o-o o o-o-o-o-o-o-o-o-o-o
| | |   |   |   |
o-o o-o-o-o-o-o-o-o-o-o-o
| |   |   |   |
o-o   |   |   |   |
B-o-o-o-o-o-o-o-o-o-o-o-o

```

On a choisi d'intégrer le Donjon 8 parce que c'est un labyrinthe conçu spécifiquement pour tendre un piège. Sur cette carte, le départ (A) et l'arrivée (B) sont du même côté (à gauche), ce qui donne l'impression qu'il suffit de descendre. Sauf qu'un mur infranchissable bloque le passage direct, et la seule vraie solution est de faire un énorme détour par la droite. Le but de ce niveau est de voir ce qui se passe quand on "trompe" la logique de nos algorithmes et de tester leurs limites.

Voici ce que ce donjon nous a permis de justifier pour chaque algorithme :

+ A* : Habituellement, c'est le meilleur des trois, mais ici son intelligence se retourne contre lui. Comme son système de visée lui dit que l'arrivée est en bas à gauche, il va refuser d'aller vers la droite (car mathématiquement, il s'éloigne de B). Au final, il visite absolument tous les culs de sac de gauche avant de se décider à faire le détour. Ce test prouve que lorsque la carte trompe l'A*, tous ses calculs de distance deviennent un poids qui ralentit énormément le programme.

+ BFS : Comme le BFS cherche de manière "bête" étape par étape sans essayer de deviner où est la sortie, il n'est pas du tout perturbé par le faux raccourci. Il va tester les mêmes impasses que l'A*, mais comme son fonctionnement est basique (il ajoute juste les cases dans une liste d'attente sans faire de maths), l'ordinateur le calcule beaucoup plus vite. Ce donjon nous permet de montrer que dans des situations comme ça, une recherche simple peut battre une recherche plus poussée.

+ DFS : Ce donjon confirme une fois de plus son point faible. En voyant un chemin qui descend vers la cible, le DFS va foncer tête baissée au fond de chaque impasse. À chaque fois qu'il tape un mur, il doit faire tout le chemin en sens inverse pour essayer une autre route. Ça nous prouve que s'il y a beaucoup de culs de sac dans un labyrinthe, sa méthode de chercher le plus loin possible lui fait perdre un temps fou.

3.2. Résultats

	DFS			BFS			A*		
Donjon	Solution	Steps	Temps	Solution	Steps	Temps	Solution	Steps	Temps
1	[SOUTH, EAST, NORTH, NORTH, EAST, SOUTH, EAST, SOUTH]	9	47	[EAST, EAST, EAST, SOUTH]	9	36	[EAST, EAST, EAST, SOUTH]	7	152
2	[WEST, WEST, NORTH, NORTH, EAST, EAST, EAST, EAST, EAST, SOUTH, SOUTH, WEST]	13	75	[EAST, NORTH, NORTH, EAST, EAST, SOUTH, SOUTH, WEST]	21	99	[EAST, NORTH, NORTH, EAST, EAST, SOUTH, SOUTH, WEST]	19	190
3	[WEST, WEST, WEST, SOUTH, EAST, EAST, EAST, EAST, EAST, EAST, NORTH, EAST, EAST]	28	66	[EAST, SOUTH, EAST, EAST, NORTH, EAST, EAST]	23	67	[EAST, SOUTH, EAST, EAST, NORTH, EAST, EAST]	11	33
4	[EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST]	15	19	[EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST]	43	106	[EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST, EAST]	15	28
5	[SOUTH, SOUTH, SOUTH, SOUTH, EAST, EAST, EAST, NORTH, NORTH]	10	9	[SOUTH, SOUTH, SOUTH, SOUTH, EAST, EAST, NORTH, NORTH]	20	23	[SOUTH, SOUTH, SOUTH, SOUTH, EAST, EAST, NORTH, EAST, NORTH]	20	37
6	[WEST, WEST, WEST, NORTH, WEST, WEST, WEST, SOUTH, SOUTH, WEST, WEST, WEST, WEST, WEST, NORTH, NORTH, WEST, WEST, SOUTH, SOUTH, SOUTH, EAST, SOUTH, EAST, EAST, SOUTH, SOUTH, EAST, NORTH, NORTH, NORTH, EAST, EAST, SOUTH, SOUTH, WEST, SOUTH, SOUTH, SOUTH, EAST, EAST, EAST, EAST, NORTH, NORTH, WEST, NORTH, EAST, EAST, EAST, NORTH, EAST, NORTH, NORTH, EAST, SOUTH, EAST, EAST, SOUTH, EAST, SOUTH, WEST, SOUTH]	353	8653	[WEST, WEST, SOUTH, EAST, EAST, EAST, EAST, EAST, NORTH, WEST]	85	507	[WEST, WEST, SOUTH, EAST, EAST, EAST, EAST, EAST, NORTH, WEST]	20	140

[illegible]

	SOUTH, SOUTH, EAST, EAST, EAST, EAST, NORTH, NORTH, EAST, NORTH, EAST, SOUTH, EAST, NORTH, EAST, EAST, EAST, SOUTH, SOUTH, WEST, SOUTH, SOUTH, SOUTH, EAST, EAST, EAST, SOUTH, EAST, EAST, SOUTH, EAST, SOUTH, EAST, SOUTH, SOUTH, EAST, EAST, EAST, EAST, NORTH, EAST, EAST, NORTH, EAST, EAST, NORTH, EAST, SOUTH, EAST, SOUTH, SOUTH, EAST, NORTH, EAST, NORTH, NORTH, NORTH, NORTH, WEST, WEST, SOUTH, WEST, NORTH, WEST, WEST, SOUTH, WEST, WEST, SOUTH, WEST, WEST, NORTH, NORTH, WEST, SOUTH, SOUTH, SOUTH, SOUTH, WEST, NORTH, NORTH, NORTH, NORTH, NORTH, EAST, NORTH, EAST, EAST, NORTH, NORTH, NORTH, EAST, EAST, EAST, SOUTH, SOUTH, EAST, EAST, EAST, NORTH, EAST, NORTH, EAST, EAST, SOUTH, EAST, SOUTH, SOUTH, WEST, NORTH, WEST, SOUTH, SOUTH, EAST, EAST, SOUTH, SOUTH, SOUTH, WEST, SOUTH, SOUTH, EAST, SOUTH]								
8	[SOUTH, EAST, SOUTH, WEST, SOUTH, EAST, EAST, EAST, EAST, SOUTH, EAST, SOUTH, EAST, EAST, EAST, SOUTH, WEST, SOUTH, EAST, EAST, EAST, EAST, SOUTH, SOUTH, SOUTH, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST]	106	509	[SOUTH, EAST, SOUTH, SOUTH, EAST, EAST, EAST, SOUTH, EAST, SOUTH, EAST, EAST, EAST, SOUTH, SOUTH, EAST, EAST, EAST, SOUTH, SOUTH, SOUTH, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST]	108	487	[SOUTH, EAST, SOUTH, SOUTH, EAST, SOUTH, EAST, EAST, EAST, SOUTH, EAST, EAST, EAST, SOUTH, SOUTH, EAST, EAST, EAST, SOUTH, SOUTH, SOUTH, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST, WEST]	108	815

3.2. Analyse

Donjon	Explications des résultats obtenus
--------	------------------------------------

1	Meilleur Algorithme : BFS est le plus rapide au chronomètre. Explication : BFS affiche le meilleur temps (36 ms), mais les délais sont trop courts pour être de vraies métriques. A* et BFS sortent le chemin optimal (7 à 9 cases). Le DFS (47 ms) est derrière. Différence : A* met bien plus de temps (152 ms) que BFS et DFS, car contrairement à ces algorithmes, lui il perd son temps à calculer, mais c'est ce qui lui permet de le faire en 7 steps seulement. Impact de la forme : C'est une forme basique.
2	Meilleur Algorithme : DFS . Explication : DFS affiche le meilleur temps (75 ms) et est celui qui trouve le chemin en moins d'étapes. A* et BFS trouvent la bonne solution. Différence : Différence de qualité de chemin. Le DFS gagne en temps et en steps, mais A* et BFS restent les seuls algorithmiquement fiables pour la solution. Impact de la forme : L'espace est confiné mais possède plusieurs routes. Le DFS a pris la première route disponible dans son code (l'Ouest) et a continué jusqu'à la sortie par chance, sacrifiant l'optimisation du chemin pour la vitesse.
3	Meilleur Algorithme : A*. Explication : A* est à la fois le plus rapide (33 ms) et le plus efficace (11 steps). Le DFS (66 ms) perd son avantage de temps car il s'égare totalement et produit un chemin de 28 cases. Différence : L'écart des cases visitées confirme que le DFS ne cherche pas le chemin le plus court. L'A* isole la cible. BFS est meilleur que DFS également (67 ms pour 33 steps) Impact de la forme : Les petits couloirs et les premiers virages annulent complètement la vitesse aveugle du DFS.
4	Meilleur Algorithme : DFS. Explications : A* file droit au but (15 steps) en ignorant les impasses grâce à son heuristique. DFS trouve aussi 15 steps en 19 ms, mais c'est un coup de chance : la première direction qu'il a testée (Est) était la bonne. BFS quant à lui, perd son temps à fouiller chaque impasse latérale (43 steps). Différence : A* et DFS divisent la charge mémoire par 3 comparé au BFS. Impact de la forme : DFS est grandement avantage par la forme du donjon, car son fonctionnement porte sur le fait de choisir 1 voisin parmi tous les voisins du noeud actuel, cela veut dire que moins il y a de voisins (en occurrence 2 à chaque fois dans le Donjon4), plus il a de chances de trouver le bon chemin.
5	Meilleur Algorithme : DFS Explication : DFS sort un temps record de 9 ms pour 10 steps. Il est le plus rapide, mais pas forcément le plus intelligent. BFS est le 2ème meilleur, il le fait en 23 ms pour 20 steps. A* le fait en 37 ms pour 20 steps Différence : A* met trop de temps pour faire ses calculs Impact de la forme : DFS est grandement avantage par la forme du donjon, qui est petit, et ne comporte pas beaucoup de noeuds voisins

6	Meilleur Algorithme : A*. Explications : A* trouve la sortie en 140 ms pour 20 steps grâce au calcul de Manhattan. BFS trouve la route mais met beaucoup d'étapes (85 steps). DFS est inefficace (353 steps, presque 10 secondes) car il explore chaque cul-de-sac jusqu'au bout avant de reculer. Différence : A* consomme moins de mémoire que BFS. DFS est bien moins compétent pour ce donjon. Impact de la forme : La densité des impasses détruit le parcours en profondeur (DFS) et prouve l'utilité du guidage de A*.
7	Meilleur Algorithme : A*. Explication : A* coupe la carte en diagonale (201 steps). Le surcoût de son calcul mathématique est rentabilisé. BFS met quasiment 18 secondes pour 528 steps pour traiter 90% d'informations inutiles. Différence : A* est 3x plus rapide (5.7s) que le BFS (17.9 s) et 2x plus rapide que le DFS (11.7 s) Impact de la forme : Le Donjon est très vaste et ne contient aucun obstacle, le calcul de A* est donc correct
8	Meilleur Algorithme : BFS. Explication : L'heuristique pousse A* vers le mur. A* va dans les impasses et génère autant d'étapes que le BFS (108 steps). Sans avantage, les calculs mathématiques de A* ralentissent l'exécution, c'est pourquoi ça prend 815 ms. Différence : À nombre d'étapes égal, les opérations basiques du BFS se révèlent environ 2x fois plus rapides (487ms) que les calculs lourds de l'A* (815ms). Il est également plus rapide que DFS car celui-ci est obligé de faire le chemin retour à chaque fois, revenir sur ses pas, ce qui fait qu'il perd du temps. Impact de la forme : L'obstacle coupe le chemin optimal et détruit donc la fiabilité de la distance de Manhattan.

Conclusion Générale sur les résultats obtenus

L'analyse des 8 donjons montre qu'il n'existe pas d'algorithme de recherche qui soit toujours le meilleur. Les performances, que ce soit en temps d'exécution ou en nombre d'étapes, dépendent surtout de la structure du labyrinthe.

- DFS (Depth-First Search) : Le DFS n'est pas fiable pour trouver le chemin le plus court. Même s'il est parfois rapide (comme dans les donjons 2, 4 et 5), c'est surtout dû au hasard : l'ordre dans lequel il explore les chemins correspondait simplement à la bonne direction.

En revanche, dès que le labyrinthe devient plus complexe (beaucoup d'impasses ou de grands espaces ouverts), ses performances chutent fortement. À cause de son fonctionnement (pile / LIFO), il fait beaucoup de retours en arrière, ce qui le rend inefficace. Il peut aussi produire des chemins inutiles ou trop longs.

- BFS (Breadth-First Search) : Le BFS garantit toujours le chemin le plus court, ce qui est son principal avantage.

Cependant, il explore tout autour de manière uniforme (file / FIFO), ce qui devient un problème dans les grands espaces ouverts. Il consomme alors beaucoup de mémoire pour analyser des zones inutiles.

Il reste intéressant dans certains cas particuliers, notamment quand il y a beaucoup de pièges ou de fausses pistes, car il ne se fait pas "tromper".

- A* (Astar) : L'algorithme A* est globalement le plus performant. Grâce à l'heuristique (distance de Manhattan), il oriente la recherche vers la cible, ce qui réduit fortement le nombre de cases explorées.

Il est donc très efficace dans les labyrinthes complexes et évite aussi le problème des grands espaces vides.

Sa limite apparaît quand le chemin direct est bloqué : dans ce cas, il peut perdre du temps à explorer des impasses à cause de son calcul.

Bilan général :

Au final, le choix d'un algorithme ne dépend pas seulement de sa vitesse, mais aussi de sa capacité à gérer l'espace et la mémoire.

- Astar est le meilleur choix par défaut
- BFS peut servir d'alternative fiable
- DFS est à éviter pour ce type de problème

4. Précisions en cas de variation du *starting code* (*)

(*) SEULEMENT SI le code ne suit pas à la lettre les éléments fournis dans le sujet (classes, attribut, signatures de méthodes, etc.). Précisez alors par classe modifiée ou ajoutée en quoi cela varie du sujet et pourquoi.

Classe X (modification / ajout)

- ajout de ...
- suppression de ...
- ...